

Prompt Fundamentals on Real Tasks v2:

Task: Design the backend architecture for an AI-powered university chatbot using Retrieval-Augmented Generation (RAG).

<i>Version</i>	<i>Task</i>	<i>Output</i>	<i>Note and explanation</i>
01 (basic)	Build the backend for the AI chatbot for university students using RAG.	Here's the plan, no code yet: 1. Framework: FastAPI for the API layer (async, auto-generated docs at /docs). 2. Embeddings: sentence-transformers (all-MiniLM-L6-v2), running locally — no external API cost just to embed text. 3. Vector store: ChromaDB, persisted to a local folder — zero extra infra to stand up, swappable later for pgvector/Pinecone if you outgrow it. 4. Ingestion pipeline: upload endpoint accepts PDF/DOCX/TXT/MD → extract text → chunk it (paragraph-aware, with overlap) → embed → store with metadata (course code, department, category). 5. Retrieval: on each question, embed the query, similarity-search Chroma, optionally filtered by course/department. 6. Generation: send retrieved chunks + conversation history to Claude with a system prompt that forces it to answer only from context, cite sources, and admit when it doesn't know rather than guessing. 7. Conversation memory: simple session-keyed history (in-memory to	Claude produced a generic RAG backend consisting of the essential components such as FastAPI, ChromaDB, embeddings, document ingestion, retrieval, generation, memory, and API routes. While technically correct, the architecture lacked implementation order, optimization strategies, and justification for the selected technologies.

		start; swappable for Redis/DB later). 8. Routes: /chat (ask + clear session), /documents (upload, list, delete), /health.	
02 (role assignment)	Act as a senior AI software engineer experienced in Retrieval-Augmented Generation (RAG), Python and LiveKit to build the backend for the AI chatbot for university students.	Here's a high-level roadmap for building this: - Requirements & scope - Define use cases (course FAQs, admissions, campus services, academic advising, etc.) - Decide interaction mode: text chat, voice, or both (LiveKit suggests voice/real-time is a priority) - Data collection & preprocessing - Gather source docs: handbooks, course catalogs, FAQs, policies, calendars - Clean, chunk, and structure the text (semantic or fixed-size chunking) - Embedding & vector store - Choose an embedding model (e.g., OpenAI, Cohere, or open-source like BGE/E5) - Pick a vector DB (Pinecone, Weaviate, Qdrant, or pgvector) and index chunks with metadata (source, department, date) - Retrieval pipeline	The output became more implementation-oriented by introducing a structured development roadmap. Claude expanded the backend beyond individual components by identifying project phases such as requirements gathering, evaluation, deployment, and scaling. It also suggested multiple technology options rather than a single solution, making the architecture more realistic.

		• Build query embedding + similarity search • Add re-ranking (optional) and metadata filtering (e.g., by faculty/program) 5. LLM integration • Choose an LLM (Claude, GPT, or open-source) for generation • Design the prompt template: system instructions, retrieved context, guardrails against hallucination • Add citation/source attribution back to students 6. LiveKit integration • Set up LiveKit server/room for real-time voice or chat sessions • Build an agent (LiveKit Agents framework) that streams STT → RAG pipeline → LLM → TTS (if voice) • Handle turn-taking, interruptions, and session state 7. Backend architecture • API layer (FastAPI is common) to orchestrate retrieval + generation • Session/conversation memory management (short-term context + optional long-term per-user history) • Async/streaming responses for low latency 8. Guardrails & evaluation	
--	--	--	--

		• Add content filters, scope limiting (stay on-topic for university queries) • Build eval set of Q&A pairs to test retrieval accuracy and answer quality • Logging/analytics for query patterns and failure cases 9. Deployment & scaling • Containerize (Docker), deploy LiveKit + backend services • Add caching, rate limiting, monitoring (latency, token usage, retrieval hit-rate) 10. Iteration • Collect feedback from real student queries, expand knowledge base, tune chunking/retrieval params	
03 (context and motivation)	Act as a senior AI software engineer experienced in Retrieval-Augmented Generation (RAG), Python and LiveKit. I am building an AI-powered voice chatbot for my internship's capstone project. The chatbot will answer questions in voice and text format and the input will be in the same two methods. The information is to be scraped from	Here's a high-level roadmap for this build: 1. Data Collection & Preprocessing • Scrape the university website (Scrapy or Playwright for JS-heavy pages, respecting robots.txt) • Clean HTML → plain text, preserve source URLs/metadata (page title, section, last-updated date) for citations • Chunk text semantically (e.g., 300–500 tokens with overlap) — keep chunk-to-source mapping	The backend architecture became tailored to the project requirements . Claude incorporated website scraping, citation handling, LiveKit voice integration, latency optimization, and hybrid retrieval techniques. The output focused on solving the specific

	the university's website with quick response time. The responses must contain citations as well.	2. Embedding & Vector Store • Generate embeddings (OpenAI text-embedding-3-small, or open-source like BGE/E5 if cost-sensitive) • Store in a vector DB with metadata filtering — Qdrant, Weaviate, or Pinecone (Qdrant is a solid free/self-hosted choice for a capstone) • Set up incremental re-scraping/re-indexing (cron job) so content stays fresh 3. Retrieval + Generation Pipeline • Hybrid search (dense + BM25/keyword) improves recall on names, course codes, etc. • Re-rank top-k chunks (Cohere rerank or cross-encoder) for precision • Prompt LLM with retrieved chunks + explicit instruction to cite source URLs/titles inline • Structure output so citations are parseable (e.g., [1], [2] mapped to a source list) — critical for voice, since you'll need to convert citation markers into spoken-friendly form or a side-channel UI display 4. Real-Time Voice Layer (LiveKit) • LiveKit Agents framework (Python) for the pipeline: STT → LLM/RAG → TTS • STT: Deepgram or LiveKit's built-in streaming STT for low latency	challenges of an AI-powered university assistant rather than describing a general RAG system.
--	---	---	--

		• TTS: ElevenLabs, Cartesia, or Deepgram Aura — prioritize streaming/low-latency options • Use LiveKit's VoicePipelineAgent (or newer Agent class) to wire STT → your RAG function → TTS, with interruption handling • For text input, reuse the same RAG backend via a parallel text/data channel or REST endpoint 5. Latency Optimization • Stream LLM tokens into TTS as they generate (don't wait for full response) • Cache frequent queries/embeddings • Keep retrieval top-k small (3–5 chunks) and reranking fast • Run vector DB and LLM calls concurrently where possible 6. Citation Handling for Dual Modality • Text responses: inline markdown links/footnotes • Voice responses: spoken attribution ("According to the admissions page...") plus optionally push full citations to a UI panel via LiveKit data messages 7. Orchestration & API Layer • FastAPI backend exposing both the RAG endpoint (text) and LiveKit room/token generation	
--	--	--	--

		• LiveKit server (self-hosted or Cloud) manages the real-time voice session 8. Evaluation • Test retrieval accuracy (does it pull the right page?) and citation faithfulness (does the answer match the cited source?) • Track end-to-end latency (STT + retrieval + LLM + TTS) as your key capstone metric	
04 (examples)	Act as a senior AI software engineer experienced in Retrieval-Augmented Generation (RAG), Python and LiveKit. I am building an AI-powered voice chatbot for my internship's capstone project. The chatbot will answer questions in voice and text format and the input will be in the same two methods. The information is to be scraped from the university's website with quick response time. The responses must contain citations as well. For example, if a student asks, "When is the	Here's a high-level roadmap for building this: 1. Data Collection & Preprocessing • Scrape the university website (Scrapy or BeautifulSoup + requests) — pages like admissions, academics, fees, deadlines • Clean HTML → plain text, preserve structure (headings, tables) since deadlines/dates often live in tables • Chunk text (400–800 tokens, with overlap) — keep metadata per chunk: source URL/filename, page title, section heading, last-scraped date 2. Embedding & Vector Store • Embed chunks (OpenAI text-embedding-3-small, or a local model like bge-small if cost/latency matters)	Providing an example response influenced the backend design itself. Claude added mechanisms to enforce consistent citation formatting, structured responses, metadata preservation, and post-processing to guarantee that every answer follows the required response style. The backend now directly supports the expected user experience.

	admission deadline?”, the expected answer should be something like, “The admission deadline is 31 August 2026 according to the Admission Office. Source: admission.html” The chatbot must follow this with style for the responses.	• Store in a vector DB — Chroma or Qdrant for a capstone (lightweight, fast); Pinecone if you want managed/cloud • Index metadata alongside vectors so you can cite source filenames later 3. Retrieval + Generation (RAG core) • On query: embed → top-k similarity search (k=3–5) → optional rerank (cross-encoder) for precision • Build a prompt template that forces the LLM to answer <i>only</i> from retrieved chunks and to append a citation using the chunk's source metadata • Use a fast LLM (GPT-4o-mini, Claude Haiku, or similar) for low latency 4. Response Formatting • Enforce your citation style via prompt instructions + few-shot examples (your "Source: admission.html" pattern) • Post-process to strip filler and guarantee citation is always present (fallback: append source even if model forgets) 5. Voice + Text I/O via LiveKit • LiveKit Agents framework: STT (Deepgram/Whisper) → your RAG pipeline → TTS (ElevenLabs/Cartesia/PlayHT) • Same RAG backend serves both text and voice	
--	---	--	--

		input — just swap the I/O layer - For voice, keep citations spoken naturally ("...according to the Admission Office") but also push full text+source to a chat/UI panel simultaneously 6. Latency Optimization - Cache embeddings/index (don't rescrape live) - Use streaming LLM responses + streaming TTS to reduce perceived latency - Keep vector DB local/in-memory if dataset is small 7. Scheduled Re-scraping - Cron job (weekly/daily) to re-scrape and re-embed changed pages, so deadlines stay current	
05 (output structure)	Act as a senior AI software engineer experienced in Retrieval-Augmented Generation (RAG), Python and LiveKit. I am building an AI-powered voice chatbot for my internship's capstone project. The chatbot will answer questions in voice and text format and the input will be in the same two methods. The information is to	Here's a high-level roadmap I'd follow for this project: 1. Data Acquisition (Scraping) - Crawl the university website (start with sitemap.xml if available, else BFS crawler with scrapy or playwright for JS-heavy pages) - Respect robots.txt, rate-limit requests, dedupe URLs - Clean HTML → extract main content (strip nav/footer/ads) using trafilatura or BeautifulSoup + heuristics	The architecture became significantly more detailed. Claude expanded the data pipeline by introducing semantic chunking, metadata indexing, hybrid search, reranking, streaming responses, evaluation metrics, and

	be scraped from the university's website with quick response time. The responses must contain citations as well. For example, if a student asks, "When is the admission deadline?", the expected answer should be something like, "The admission deadline is 31 August 2026 according to the Admission Office. Source: admission.html" The chatbot must follow this with style for the responses with the following format and headings: Question asked, Context, Response, Sources.	• Store raw + cleaned text with metadata: {url, title, last_scraped, section} 2. Chunking & Preprocessing • Split by semantic units (headings, paragraphs) rather than fixed-length — use RecursiveCharacterTextSplitter or heading-aware chunking • Keep chunk size ~300–500 tokens with slight overlap • Attach source metadata (URL/filename) to every chunk — this is what powers your citations later 3. Embedding & Vector Store • Choose an embedding model (OpenAI text-embedding-3-small, or open-source like bge-small for cost/latency) • Store in a fast vector DB: Qdrant, Weaviate, or Chroma (Qdrant is a solid pick for low-latency local/self-hosted) • Index metadata fields for filtering (e.g., by department/section) 4. Retrieval Pipeline • Hybrid search (dense + BM25/keyword) improves recall for exact terms like "deadline," "GPA," etc. • Add a reranker (e.g., bge-reranker or Cohere rerank) to boost precision before passing to LLM	guardrails. The backend now addressed both system performance and answer quality, producing a more complete production-style design.
--	---	--	---

		• Return top-k chunks with their source metadata intact 5. LLM Response Generation • Prompt template enforcing: Question / Context / Response / Sources structure • Instruct the LLM to only answer from retrieved context (reduce hallucination) and cite the exact source filename/URL per claim • Use a fast model for low latency (e.g., a smaller/distilled model, or streaming responses from a larger one) 6. Voice Layer (LiveKit) • Use LiveKit Agents framework — it already has plugins for STT (Deepgram/Whisper), LLM, and TTS (ElevenLabs/Cartesia/PlayHT) in a pipeline • Wire your RAG pipeline in as the "LLM" step (custom function) inside the LiveKit Agent • Enable interruption handling + streaming TTS so responses feel real-time • Support both voice and text input in the same LiveKit room (data channel for text, audio track for voice) 7. Latency Optimization • Stream tokens from LLM → TTS incrementally	
--	--	---	--

		(don't wait for full response) • Cache frequent queries/embeddings • Use async/concurrent retrieval + generation where possible • Keep vector DB and LLM inference geographically close (same region/host) 8. Evaluation & Guardrails • Test retrieval accuracy (does it fetch the right chunk?) • Test citation faithfulness (does the cited source actually support the claim?) • Add fallback: "I couldn't find this on the university website" when confidence/similarity is low • Periodic re-scraping to keep content fresh	
06 (step decomposition)	Act as a senior AI software engineer experienced in Retrieval-Augmented Generation (RAG), Python and LiveKit. I am building an AI-powered voice chatbot for my internship's capstone project. The chatbot will answer questions in voice and text format and the input will be in the same two methods. The	Claude: Question asked "Act as a senior AI software engineer... just give me the summary of the steps you'll use" — a request for the high-level plan/methodology I'll follow to design their RAG-based voice chatbot for their university capstone project (LiveKit + voice/text I/O + citation-backed answers). Context This is a planning-phase request, not the final architecture yet. The user wants to see <i>how</i> I'll approach the problem (requirements → backend	Claude first analysed the system requirements , backend services, risks, recommendations, and finally the architecture. This resulted in a more systematic design that justified each technical decision before implementation. The

	information is to be scraped from the university's website with quick response time. The responses must contain citations as well. For example, if a student asks, "When is the admission deadline?", the expected answer should be something like, "The admission deadline is 31 August 2026 according to the Admission Office. Source: admission.html" The chatbot must follow this with style for the responses with the following format and headings: Question asked, Context, Response, Sources. Before answering the system must: 1. identify the system requirements. 2. Assess the required backend services. 3. Catalog possible risks and	assessment → risk cataloging → recommendations → final backend design) before I commit to a concrete tech stack. I'm summarizing my working methodology, not building the system yet. Response Here's the summary of steps I'll follow, in order: 1. Identify System Requirements • Functional: voice-in/voice-out, text-in/text-out, low-latency response, citation-grounded answers, structured output format (Question/Context/Response/Sources) • Non-functional: response time targets (STT→LLM→TTS round trip), concurrent user load, data freshness (how often the university site changes), accuracy/hallucination tolerance • Data scope: which university domains/pages are in scope for scraping (admissions, academics, fees, etc.) 2. Assess Required Backend Services • Voice pipeline: LiveKit Agents (orchestration), STT engine (Deepgram/Whisper), TTS engine (ElevenLabs/Cartesia/PlayHT), VAD/turn-detection • RAG pipeline: scraper/crawler, chunking	output also identified deployment risks such as hallucinations, stale data, latency, and scraping failures together with mitigation strategies, making the final backend considerably more robust, although the response became longer due to the additional reasoning.
--	--	--	--

	the optimal route. 4. Provide possible recommendations. 5. Produce the final backend.	+ embedding model, vector DB (Pinecone/Qdrant/Weaviate/pgvector), retriever, re-ranker (optional) • LLM: response generation model (with citation-aware prompting) • Orchestration/API layer: FastAPI or similar to glue LiveKit ↔ RAG ↔ LLM • Storage: metadata store to map chunks back to source URLs for citations 3. Catalog Possible Risks and Optimal Route • Latency risk (voice needs sub-2s response) → route: streaming STT/LLM/TTS, small/fast embedding model, caching frequent queries • Stale data risk (site changes) → route: scheduled re-crawl + incremental indexing • Hallucination/no-citation risk → route: strict RAG-only answering, refuse/fallback when retrieval confidence is low • Scraping fragility (site structure changes, JS-rendered pages, robots.txt) → route: resilient scraper (Playwright fallback), monitoring/alerts • Cost risk (TTS/STT + LLM calls at scale) → route: model tiering, caching, self-hosted STT/TTS if budget-constrained 4. Provide Recommendations • Best-fit stack given the constraints (I'll present 1–	
--	---	--	--

		2 concrete options per layer with tradeoffs, not just one forced choice) • Recommended chunking/citation strategy so every answer can be traced to a source file/URL • Recommended fallback behaviour for out-of-scope or low-confidence questions 5. Produce the Final Backend • A concrete, justified architecture diagram + component list + data flow (scrape → index → query → retrieve → generate → cite → speak/display), followed by implementation-ready details (folder structure, key libraries, pseudocode/config) once you confirm direction. Sources N/A — this is a proposed methodology, not a factual/citable answer. ChatGPT: Here's the high-level process I would follow before building the backend. 1. Identify the system requirements ○ Define functional requirements (voice + text input/output, RAG, citations, response format).	
--	--	--	--

		○ Define non-functional requirements (low latency, scalability, accuracy, maintainability, security). ○ Identify the technologies required (Python, LiveKit, vector database, LLM, embedding model, scraper). 2. Assess the required backend services ○ Design the backend architecture. ○ Identify each service: ▪ Web scraper ▪ Document processing ▪ Embedding generation ▪ Vector database ▪ Retrieval pipeline ▪ LLM generation ▪ Citation engine ▪ LiveKit voice service ▪ REST/WebSocket API ▪ Logging and monitoring 3. Catalog possible risks and determine the optimal approach ○ Evaluate risks such as: ▪ Hallucinations ▪ Outdated website data	
--	--	--	--

		▪ Poor retrieval quality ▪ Slow response times ▪ Citation inaccuracies ▪ Voice transcription errors ○ Select the best architecture to mitigate these risks (e.g., Hybrid RAG with semantic search and metadata filtering). 4. Provide recommendations ○ Recommend the most suitable libraries and frameworks. ○ Recommend the scraping strategy. ○ Recommend the vector database. ○ Recommend embedding and LLM models. ○ Recommend deployment and scaling strategies. ○ Recommend evaluation metrics and testing procedures. 5. Produce the final backend ○ Design the complete backend architecture. ○ Implement the scraping and indexing pipeline. ○ Build the RAG retrieval pipeline. ○ Integrate the LLM with citation support.	
--	--	--	--

		○ Integrate LiveKit for voice interaction. ○ Build APIs for frontend communication. ○ Add logging, caching, error handling, and monitoring. ○ Test and optimize for response speed and accuracy. The result is a complete RAG backend that supports both text and voice conversations, retrieves information from the university website, generates grounded answers with citations, and formats every response as: • Question Asked • Context • Response • Sources	
--	--	--	--

Cross-model comparison:

Claude produced a more comprehensive and technical backend architecture with the appropriate explanations and needed justifications that matched the given requirements. However, its responses were longer. ChatGPT generated a clear and well-structured output that used major backend components however it provided less technical and fewer implementation details.

Final template:

Act as a senior AI Software Engineer experienced in Retrieval-Augmented Generation (RAG), Python, modern backend development, vector databases, and AI application architecture.

I am developing an AI-powered [application type] for [target users]. The application should answer questions using information retrieved from [knowledge source] while maintaining [performance requirement]. Responses should include [citation or reference requirement] and support [text, voice, or both] interaction.

For example:

User's question:

"[Example user question]"

Expected response:

"[Example answer]"

Source:

"[Example citation]"

Design the backend so that all responses follow this style.

Return your response using the following headings:

- Question Asked
- Context
- Response
- Sources

Before producing the solution, complete the following steps:

1. Identify the system requirements.
2. Assess the required backend services.
3. Identify potential risks and limitations.
4. Recommend the most suitable architecture, technologies, and implementation approach.
5. Produce the complete backend architecture with a clear explanation of the data flow and major components.